

PySpark Interview Q&A; Guide

Core & Advanced Concepts — Lazy Evaluation, DataFrames, Optimization, Shuffles, Joins, UDFs, Streaming and More

This guide compiles the most important PySpark questions — from the single most fundamental concept (lazy evaluation) to advanced production topics (Catalyst, AQE, shuffles, joins, Pandas UDFs, memory tuning, and structured streaming) — each with a detailed explanation.

Section 1: The Core Concept — Lazy Evaluation

Q1. What is lazy evaluation in Spark, and why is it considered the single most important concept in PySpark?

Lazy evaluation means that when you write a transformation (map, filter, select, groupBy, withColumn, etc.), Spark does **not** execute it immediately. Instead, Spark simply records the operation and builds up a **logical plan** — a Directed Acyclic Graph (DAG) describing the sequence of operations that will eventually need to run.

The computation is only triggered when an **action** is called (count(), collect(), show(), write, take(), etc.). At that point, Spark's Catalyst optimizer analyzes the entire accumulated plan — not just one step at a time — and decides the most efficient way to execute it as a whole.

This is the most important concept because virtually every other capability of Spark (fault tolerance via lineage, query optimization, efficient cluster scaling, avoiding unnecessary data movement) is a direct consequence of this lazy, plan-then-execute design. Developers who don't understand it commonly write inefficient code — calling actions too early, triggering unnecessary shuffles, or materializing huge intermediate results that were never necessary.

```
# These are just building a plan – nothing runs yet
df = spark.read.parquet("huge_data")
filtered = df.filter(df.age > 30)
grouped = filtered.groupBy("city").agg(...)

# This action triggers the entire optimized computation
result = grouped.count() # or .show(), .write..., etc.
```

Q2. What is the difference between a transformation and an action? Give examples of each.

Transformations are lazy operations that describe how a dataset should be changed, but produce no immediate computation — they just extend the DAG. Examples: map, filter, select, groupBy, withColumn, join, distinct, orderBy.

Actions are eager operations that force Spark to actually execute the recorded plan and return a concrete result (to the driver, to storage, or to the console). Examples: count(), collect(), show(), take(n), write.parquet(...), foreach().

Tip: A quick way to remember it: transformations answer 'what should happen', actions answer 'do it now'.

Q3. What practical benefits does the lazy transformation/eager action design give Spark?

Four major benefits flow directly from this design:

- **Optimization** — the Catalyst optimizer can see the whole pipeline at once and apply rules like predicate pushdown, operation reordering, and physical-plan selection, instead of optimizing operations in isolation.
- **Avoiding unnecessary work** — Spark can skip computing columns/rows that are never needed for the final action, and avoid unneeded data movement across the cluster.

- **Fault tolerance via lineage** — because Spark knows every transformation that produced a dataset, it can recompute only the lost partitions if a node fails, instead of needing to replicate all data upfront.
- **Efficient scaling** — the optimized physical plan can be distributed across a cluster far more efficiently than a naive, step-by-step eager execution would allow.

Section 2: Supporting Core Concepts

Q4. What is an RDD, and why does modern PySpark recommend DataFrames over RDDs?

An **RDD (Resilient Distributed Dataset)** is Spark's original core abstraction: an immutable, partitioned collection of objects distributed across the cluster, with fault tolerance built in through lineage tracking.

Modern guidance is to prefer **DataFrames** for almost all work. DataFrames are built on top of RDDs but add a structured, higher-level API — similar to a distributed pandas or SQL table — which lets Spark apply the Catalyst optimizer and the Tungsten execution engine. RDDs, being untyped collections of arbitrary objects, don't give Spark this structural information, so it can't optimize them nearly as effectively.

Q5. What is a DataFrame in PySpark?

A DataFrame is a higher-level, structured API that represents distributed data organized into named columns, conceptually similar to a table in a relational database or a pandas DataFrame — except it is distributed across a cluster. It is the primary API recommended for use in PySpark, since it benefits from Catalyst's query optimization and Tungsten's efficient execution.

Q6. What is a partition, and why does it matter for performance?

A **partition** is the unit of parallelism in Spark — the dataset is split into partitions, and each partition is processed independently by a **task**. The number and size of partitions directly determines how much parallel work Spark can do, how balanced that work is across executors, and how much data needs to move during a shuffle.

Too few partitions under-utilizes the cluster (tasks are too big, some cores sit idle); too many partitions creates excessive scheduling overhead from very small tasks. As a rule of thumb, aim for roughly 100–200 MB of data per partition.

Q7. What is the difference between narrow and wide transformations?

A **narrow transformation** (e.g., map, filter, withColumn on existing columns) can be computed on each partition independently — no data needs to move between partitions/nodes, so there is no shuffle.

A **wide transformation** (e.g., groupBy, join, repartition, distinct) requires data with the same key to be brought together, which means data must be redistributed across partitions — this is a **shuffle**, and shuffles are expensive because they involve disk I/O, network transfer, and serialization.

The modern recommendation is to minimize wide operations wherever possible, since they are the primary source of performance bottlenecks in Spark jobs.

Q8. What is a SparkSession and why is it described as the 'unified entry point'?

SparkSession is the single, unified entry point for programming Spark since Spark 2.0 — it replaced the earlier separate contexts (SparkContext, SQLContext, HiveContext). Every PySpark program should always start by creating or retrieving a SparkSession, since it is used to read data, create DataFrames, run SQL queries, and configure runtime behavior.

Section 3: Query Optimization Internals

Q9. What is the Catalyst Optimizer?

Catalyst is Spark SQL's query optimizer. It takes the logical plan built up from your chained transformations and rewrites it into an efficient **physical plan** using both **rule-based optimization** (e.g., predicate pushdown, constant folding, column pruning) and **cost-based optimization** (choosing between alternative physical strategies, such as join algorithms, based on estimated cost).

Q10. What is the Tungsten execution engine?

Tungsten is Spark's execution engine focused on raw CPU and memory efficiency. Its key features include **whole-stage code generation** (compiling multiple operators into a single optimized Java function to avoid virtual function-call overhead), **off-heap memory management** (reducing JVM garbage-collection pressure), and **cache-aware computation** (structuring memory access patterns to work well with CPU caches).

Q11. What is Adaptive Query Execution (AQE), and what does it do at runtime?

AQE is a set of **runtime** optimizations, enabled by default in modern Spark, that adjust the physical execution plan based on actual statistics gathered while the job is running — rather than relying solely on estimates made before execution starts.

Its main capabilities include: **coalescing shuffle partitions** (merging small post-shuffle partitions to avoid excessive task overhead), **dynamically switching join strategies** (e.g., switching a sort-merge join to a broadcast join once actual data size is known), and **handling data skew** in joins by splitting oversized partitions.

Q12. How do you inspect a DataFrame's execution plan, and why would you do it?

Call `df.explain(mode="extended")` or `df.explain(True)`. This prints the parsed logical plan, analyzed logical plan, optimized logical plan, and the final physical plan. Regularly inspecting this output is how you verify that predicate pushdown happened, confirm which join strategy was chosen, and spot unexpected shuffles before a job runs at scale.

```
df.explain(mode="extended")
# or
df.explain(True)
```

Section 4: Partitioning & Data Layout

Q13. What is the difference between `repartition()` and `coalesce()`?

`repartition(n)` performs a **full shuffle** to redistribute data into exactly `n` partitions of roughly equal size — it can both increase and decrease the partition count, and is useful when you need balanced partitions after a

skewed operation.

`coalesce(n)` avoids a full shuffle by merging existing partitions together, so it is much cheaper — but it can only **decrease** the number of partitions, and the resulting partitions may be unevenly sized since it doesn't fully redistribute data.

Q14. What partitioning strategies does Spark support, and what is bucketing?

Spark supports **hash partitioning** (default for most shuffles — assigns rows to partitions based on a hash of the key), **range partitioning** (assigns contiguous key ranges to partitions, useful for sorted output), and **custom partitioners** for specialized distribution logic on RDDs.

Bucketing is a related but distinct technique, especially useful with Hive tables or Delta Lake: data is pre-sorted and split into a fixed number of buckets by a key at write time, so that later joins or aggregations on that key can avoid a shuffle entirely.

Q15. How does partition size affect parallelism and shuffle cost, and what's a good target size?

Partition size is a direct trade-off dial. Very small partitions mean many tiny tasks, and the per-task scheduling overhead starts to dominate actual compute time. Very large partitions mean too few tasks to fill the cluster's cores, and a single slow/oversized task can become a straggler that holds up the whole stage.

The document's rule of thumb is to aim for roughly **100–200 MB per partition** as a starting point, then tune based on observed task duration and shuffle size in the Spark UI.

Section 5: Shuffles & Wide Transformations

Q16. When exactly does a shuffle occur in Spark?

A shuffle occurs whenever data with the same key needs to be brought together across partitions. Common triggers are joins (non-broadcast), `groupBy/aggregations`, `repartition()`, and `distinct()`. All of these are wide transformations by definition.

Q17. Which configuration properties control shuffle behavior, and what do they do?

`spark.sql.shuffle.partitions` controls the number of partitions used when shuffling data for joins or aggregations in Spark SQL/DataFrame operations (the default is often too high or too low for a given dataset size and should be tuned).

`spark.default.parallelism` controls the default number of partitions for RDD operations and for operations like `join/reduceByKey` when no explicit partition count is given, typically outside of Spark SQL contexts.

Q18. What techniques reduce the cost of a shuffle?

Three main techniques: **broadcast joins** (send a small table to every executor so the large table never needs to be shuffled), **salting** (artificially spreading a skewed key across multiple sub-keys so no single partition/task becomes a bottleneck), and **pre-partitioning data** at write time (e.g., via bucketing) so that a shuffle isn't needed again on subsequent joins over the same key.

Section 6: Caching & Persistence

Q19. What is the difference between `cache()` and `persist()`, and what storage levels are available?

`cache()` is shorthand for `persist(StorageLevel.MEMORY_ONLY)` (or `MEMORY_AND_DISK`, depending on API/version defaults) — a convenience method with one fixed storage level.

`persist()` lets you explicitly choose a storage level, such as `MEMORY_ONLY` (fastest, but data is lost if it doesn't fit in memory), `MEMORY_AND_DISK` (spills to disk when memory is full), `DISK_ONLY` (never uses memory), or off-heap variants (stored outside the JVM heap to reduce garbage-collection pressure).

Q20. When does caching help, and when can it actually hurt performance?

Caching helps when a `DataFrame` is reused across **multiple actions** — without it, Spark would recompute the entire lineage from scratch every single time due to lazy evaluation, wasting significant work.

Caching hurts when the cached data doesn't comfortably fit in available memory: it creates **memory pressure**, forces evictions of other cached or working data, and can trigger expensive garbage collection — sometimes making a job slower than if nothing had been cached at all. Always call `unpersist()` once cached data is no longer needed, and monitor storage levels via the Spark UI's Storage tab.

Section 7: Join Strategies & Optimization

Q21. Compare broadcast joins, shuffle hash joins, and sort-merge joins.

A **broadcast (map-side) join** sends a copy of the smaller table to every executor, so the larger table can be joined locally with no shuffle at all — fastest option when one side is small enough to fit in executor memory.

A **shuffle hash join** shuffles both tables by the join key, then builds a hash table on the smaller side of each partition pair and probes it with the other side.

A **sort-merge join** shuffles both tables by the join key, sorts each partition, then merges the sorted streams together — this is Spark's default strategy for large-to-large joins because it scales well and doesn't require the join key's data to fit in memory.

Q22. What controls whether Spark automatically uses a broadcast join?

The config `spark.sql.autoBroadcastJoinThreshold` sets the maximum estimated size (in bytes) of a table for which Spark will automatically choose a broadcast join instead of a shuffle-based join. If a table's estimated size is below this threshold, Spark (or AQE at runtime) will broadcast it automatically; setting it to `-1` disables automatic broadcasting.

```
spark.conf.set("spark.sql.autoBroadcastJoinThreshold", 10 * 1024 * 1024) # 10MB
```

Q23. How do you handle data skew in joins?

Two main approaches: **salting**, where you append a random suffix to the skewed key on both sides of the join (and explode the smaller side across all salt values) so that no single key value dominates one task; and **AQE's adaptive skew join handling**, which automatically detects oversized shuffle partitions at runtime and splits them into smaller sub-partitions before completing the join.

Q24. What are semi and anti joins, and why are they useful?

A **semi join** (leftsemi) returns rows from the left table that have at least one matching row in the right table — but only columns from the left table, with no duplication even if there are multiple matches. An **anti join** (leftanti) returns rows from the left table that have **no** match in the right table.

Both are more efficient than an inner join followed by a filter/dedupe, because Spark never needs to materialize the matched columns from the right side or handle multi-match duplication — this is especially useful for existence checks and exclusion filters over large datasets.

Section 8: User-Defined Functions (UDFs)

Q25. Why are regular Python UDFs slow in PySpark?

A regular Python UDF forces Spark to serialize each row from the JVM, send it to a separate Python process, execute the function **row-by-row**, and serialize the result back to the JVM. This serialization/deserialization round-trip and the loss of Catalyst/Tungsten optimization on the UDF body make regular UDFs one of the most common sources of performance problems in PySpark code.

Q26. What are Pandas UDFs and how are they different from regular UDFs?

Pandas UDFs (vectorized UDFs) use Apache Arrow to transfer batches of data between the JVM and Python in a columnar, vectorized format instead of row-by-row, and operate on pandas Series/DataFrame batches inside Python. This dramatically reduces serialization overhead and lets you use vectorized pandas/NumPy operations, making them much faster than regular row-at-a-time UDFs.

There are several kinds: **scalar Pandas UDFs** (element-wise transform of one or more columns), **grouped map Pandas UDFs** (apply a function to each group of a groupBy and return a transformed DataFrame per group), and **aggregate Pandas UDFs** (compute a single aggregated value per group, usable in groupBy().agg()).

Tip: Whenever possible, prefer Spark SQL's built-in functions over any kind of UDF — built-ins are fully optimized by Catalyst and require no serialization at all.

Section 9: Window Functions

Q27. What are window functions used for in PySpark, and what is a frame specification?

Window functions perform calculations across a set of rows that are related to the current row — without collapsing rows the way a groupBy aggregation does. Common uses include ranking (rank, dense_rank, row_number), running totals, lag/lead (accessing previous/next row values), and moving averages.

A **frame specification** defines exactly which rows, relative to the current row, participate in the window calculation. rowsBetween defines the frame by a physical number of rows (e.g., the 2 preceding rows), while rangeBetween defines it by a value range on the ordering column (e.g., all rows within 7 days of the current row's date).

```
from pyspark.sql import Window
from pyspark.sql import functions as F
```

```
w = Window.partitionBy("city").orderBy("date").rowsBetween(-2, 0)
df.withColumn("moving_avg", F.avg("sales").over(w))
```

Q28. Why are window functions often expensive, performance-wise?

Window functions typically require the data to be **partitioned** (by the `partitionBy` columns, which involves a shuffle if not already co-located) and then **sorted** within each partition (by the `orderBy` columns). Both the shuffle and the sort add cost, so window functions over very large datasets or with high-cardinality partition keys should be used deliberately and tuned like any other wide operation.

Section 10: Memory Management & Tuning

Q29. How is executor memory divided, and what is the unified memory model?

Executor memory is split between **execution memory** (used for shuffles, joins, sorts, and aggregations) and **storage memory** (used for caching/persisting DataFrames and broadcast variables). Spark's **unified memory model** allows these two regions to borrow space from each other dynamically, up to configured limits, rather than using two rigid, statically-sized pools — improving memory utilization overall.

Q30. What is Kryo serialization, and how does it relate to off-heap memory?

Kryo is an alternative, faster and more compact serializer than Java's default serialization, and it is commonly enabled for better performance, especially with RDD operations and caching. **Off-heap memory** stores certain data structures outside the JVM heap entirely (a Tungsten feature), which reduces the amount of memory the JVM's garbage collector has to manage and helps avoid GC pauses on large executors.

```
spark.conf.set("spark.serializer", "org.apache.spark.serializer.KryoSerializer")
```

Q31. What commonly causes out-of-memory (OOM) errors in Spark jobs?

Frequent causes include: data skew (one partition/task receives a disproportionate share of data), caching too much data relative to available executor memory, very wide or deeply nested schemas that inflate row size, collecting very large results back to the driver with `collect()`, and broadcast joins where the 'small' table turns out to be too large for the configured broadcast threshold.

Q32. What is the difference between dynamic allocation and fixed executors?

With **fixed executors**, you specify a static number of executors for the entire application's lifetime, which can waste cluster resources during idle periods or bottleneck during peak load. With **dynamic allocation** enabled, Spark automatically requests more executors when there is a backlog of pending tasks and releases executors that have been idle, adapting resource usage to the actual workload over time.

Section 11: Structured Streaming (Advanced)

Q33. What is watermarking in Structured Streaming, and why is it needed?

A watermark defines how long Spark should wait for **late-arriving data** before considering a given event-time window 'closed' and safe to finalize/emit. Without a watermark, Spark would need to keep unbounded state forever, since data could theoretically arrive arbitrarily late — watermarking bounds that state growth by accepting a defined trade-off between completeness and resource usage.

```
df.withWatermark("event_time", "10 minutes")\
  .groupBy(window("event_time", "5 minutes"))\
  .count()
```

Q34. What are stateful operations like `mapGroupsWithState` / `flatMapGroupsWithState` used for?

These APIs let you maintain and update arbitrary custom state per key across streaming micro-batches — beyond what built-in aggregations support. They're used for use cases like session tracking, complex event pattern detection, or custom incremental aggregations, where you need explicit control over how state is initialized, updated, and expired for each key over time.

Q35. What are the differences between `append`, `update`, and `complete` output modes?

Append mode only outputs new rows added since the last trigger, and only supports queries where existing rows in the result table will never change (e.g., no aggregation, or aggregation with a watermark). **Update** mode outputs only the rows that were updated since the last trigger — appropriate for aggregations that can be revised. **Complete** mode outputs the entire result table on every trigger, which works for aggregations but does not scale well as the result grows large.

Q36. How does Structured Streaming achieve exactly-once semantics?

Exactly-once processing is achieved through the combination of **checkpointing** (persisting the streaming query's progress/offsets and state so it can resume exactly where it left off after a failure) and **idempotent sinks** (writing results in a way that reprocessing the same batch after a failure doesn't produce duplicate or corrupted output, e.g., via transactional writes or upserts keyed on a unique batch/record id).

Section 12: Other High-Value Advanced Topics

Q37. What are broadcast variables and accumulators, and how do they differ?

A **broadcast variable** is a read-only value efficiently distributed once to every executor (rather than being shipped with every task), used for things like small lookup tables or configuration data needed across many tasks.

An **accumulator** is a write-only (from the tasks' perspective) shared variable that tasks can only add to, and which the driver can read after an action completes — commonly used for counters or sums accumulated across distributed tasks, such as counting malformed records during a parsing job.

Q38. What advanced features do Delta Lake / Iceberg add on top of plain Spark tables?

Lakehouse formats like Delta Lake and Iceberg add ACID transactions and versioning on top of files in object storage. Key advanced features mentioned include: **time travel** (querying a table as it existed at a previous version or timestamp), **OPTIMIZE** (compacting many small files into larger ones for better read performance), **Z-ORDER** (co-locating related data across multiple dimensions on disk to speed up multi-column filtering), and **vacuum** (permanently removing old, no-longer-referenced data files to reclaim storage).

Q39. What is Spark Connect?

Spark Connect is a client-server architecture that decouples the client application from the Spark driver/cluster. A thin client can send DataFrame API calls over gRPC to a remote Spark server, which means client applications (including from languages/environments not traditionally embedded in the driver JVM) don't need direct, tightly-coupled access to cluster internals — improving stability, remote connectivity, and cross-language support.

Q40. What is barrier execution mode, and when would GPU acceleration be relevant?

Barrier execution mode ensures that all tasks in a stage start together and run concurrently as a single logical group, which is required for workloads like distributed deep learning training where individual worker processes need to communicate with each other directly (e.g., via MPI-style all-reduce) rather than running as fully independent tasks.

GPU acceleration becomes relevant for compute-heavy workloads such as machine learning training/inference embedded in a Spark pipeline; Spark's cluster manager integration can schedule and allocate GPU resources to executors/tasks that need them, though this remains a less common configuration than standard CPU-based Spark usage.

Section 13: Recommended Learning Order for Advanced Topics

Based on how these concepts build on one another, the suggested progression is:

- 1. Catalyst + explain() + AQE — understand how your code is actually optimized and executed.
- 2. Partitioning, shuffles, and join strategies — the biggest levers for real-world performance.
- 3. Caching + the unified memory model — knowing when persistence helps vs. hurts.
- 4. Pandas UDFs — for the (hopefully rare) cases where built-in functions aren't enough.
- 5. Window functions + skew handling — advanced analytics without collapsing your dataset.
- 6. Streaming state management — watermarking, stateful ops, and exactly-once semantics.

Appendix: Core Concepts Quick-Reference Table

Concept	Why It Matters	Modern Recommendation
RDD	Original core abstraction: immutable, partitioned, fault-tolerant distributed collection	Prefer DataFrames for almost all work

DataFrame	Higher-level, structured API (like a distributed pandas/SQL table)	Primary API to use in PySpark
Partitions	The unit of parallelism — data is split into partitions processed by tasks	Control them for performance (~100-200MB each)
Narrow vs Wide transformations	Narrow = no shuffle; Wide = shuffle (expensive)	Minimize wide ops when possible
SparkSession	Unified entry point	Always start here

Bottom line: Master lazy evaluation + transformations/actions (and prefer DataFrames over raw RDDs) first. Performance, fault tolerance, scaling, and optimization in PySpark all flow from that single foundation — the advanced topics above are refinements built on top of it.